Project Title

World Level Plagiarism Checker

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

Jalla Geya Reddy (2520090123)
Kamtam Priyanka (2520090001)

Team Number: 9

Section: 11

Under the guidance of

Mrs. Anitha Chekuri
Assistant Professor, CS&IT

Hyderabad-500090, Telangana, India.



K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**World Level Plagiarism Checker"** is a record of Bonafide work **Jalla Geya Reddy - 2520090123, Kamtam Priyanka - 2520090001.** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L H University. The results embodied in this report have not been copied from any other departments/University/Institute.

Jalla Geya Reddy – 2520090123

Kamtam Priyanka - 2520090001

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**World Level Plagiarism Checker"** is a bona fide work done and submitted by **Jalla Geya Reddy - 2520090123, Kamtam Priyanka - 2520090001.** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L H (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Mrs. Anitha Chekuri, Assistant Professor, CS&IT which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef,** Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Jalla Geya Reddy – 2520090123

Kamtam Priyanka – 2520090001

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

World-Level Plagiarism Checker is a Java-based document comparison system developed as a Data Structures and Algorithms -3 project. The system allows a user to upload one student document and multiple reference documents in TXT, PDF, or DOCX format and produces a ranked similarity report for each reference. Apache Tika is used to extract text from supported document formats, after which the existing Java DSA engine processes the extracted content. The project applies custom data structures including linked lists, hash sets, hash tables, and a max heap, together with algorithms such as Rabin-Karp string matching, Longest Common Subsequence (LCS), edit distance, Jaccard similarity, and synonym-based comparison. The system classifies comparisons into Exact Match, Reordered Words, Possible Paraphrase, Partial Similarity, and Low / No Match. A synonym dictionary is loaded from a local corpus to support paraphrase-oriented comparisons. The project is exposed through a Spring Boot web application with a simple browser interface, validation, structured error handling, and support for multiple reference uploads. Testing confirmed successful processing of TXT, PDF, and DOCX files, multiple references, synonym-based paraphrase detection, exact matching, and reordered-word classification. The project demonstrates how core DSA concepts can be integrated into a practical document-analysis application while keeping the detection engine independent from the web layer.

Team Number: 9

Team Members:

1. Jalla Geya Reddy, 2520090123
2. Kamtam Priyanka- 2520090001

2. INTRODUCTION

Plagiarism checking is the process of identifying similarities between a submitted document and one or more reference documents. In academic environments, a useful checker should not only identify identical text but should also provide an understandable indication of reordered or paraphrased content.

The World-Level Plagiarism Checker was developed to demonstrate how Data Structures and Algorithms can be applied to this problem. Instead of depending entirely on an external plagiarism service, the project uses an existing Java DSA engine that performs document processing and similarity analysis locally.

The system accepts a student document and multiple reference documents. Apache Tika extracts text from TXT, PDF, and DOCX files. The extracted text is normalized and tokenized before being analyzed by the plagiarism engine. Rabin-Karp is used for exact sentence matching, while LCS and Jaccard-based calculations support similarity and reordered-word analysis. Edit-distance and synonym-related logic contribute to broader comparison and paraphrase detection.

The web layer is implemented using Spring Boot. It provides file-upload endpoints, validation, structured error responses, and a browser-based interface that displays similarity percentages, match classifications, and evidence snippets.

3. LITERATURE SURVEY

1. **Turnitin:** An established academic similarity-checking platform used to identify overlap between submitted work and reference content. It represents the broader class of systems that generate similarity reports for academic integrity review. This project differs by implementing its comparison engine locally with custom DSA structures and algorithms.

2. **Grammarly Plagiarism Checker:** Grammarly provides an originality/plagiarism checking feature that compares writing against large collections of online and published content and highlights potential similarities. This demonstrates the value of presenting detected similarities clearly to users. The proposed project provides a simpler educational report with percentage, classification, and evidence.

3. **Quetext:** Quetext describes a checker using contextual analysis, fuzzy matching, and conditional scoring to identify exact and near matches. The proposed project similarly distinguishes types of similarity, but focuses on DSA techniques such as hashing, Rabin-Karp, LCS, Jaccard similarity, and heap-based ranking.

4. **Scribbr:** Scribbr provides document upload and similarity checking with support for common academic document formats and source-oriented similarity reports. This is relevant because the proposed system also accepts document files. Apache Tika extracts the text before the DSA engine performs comparison.

5. **Copyleaks:** Copyleaks provides plagiarism detection through document and API workflows and supports identical and paraphrased content detection. The proposed system adopts the comparison concept but keeps its detection logic inside the Java project for educational demonstration of DSA.

6. **Apache Tika:** Apache Tika is a content analysis toolkit capable of extracting text and metadata from many file formats. In this project it forms the document parsing layer for TXT, PDF, and DOCX files, separating file-format handling from the plagiarism algorithms.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Develop a Java-based plagiarism comparison system that can compare a student document with multiple reference documents and classify different types of textual similarity.

Problem Description: Manual comparison of documents is time-consuming and may miss reordered, partially similar, or synonym-based content. The project provides an automated DSA-based comparison workflow with a simple web interface.

Objectives:

- Develop a system that compares one student document with multiple reference documents at the same time.
- Enable users to upload TXT, PDF, and DOCX files and extract their text automatically.
- Clean and organize the extracted text so it can be compared accurately.
- Apply Rabin–Karp, LCS, and Jaccard Similarity to identify matching text and measure similarity between documents.
- Use a synonym dictionary to identify documents that may contain paraphrased content.
- Rank reference documents according to their similarity scores to help users identify the closest matches.
- Display similarity percentages, match categories, and matching text snippets through a web interface.
- Validate uploaded files and provide clear error messages when files are missing, invalid, or unsupported.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

• Computer/Laptop with a modern 64-bit processor.

• Minimum 4 GB RAM; 8 GB or more recommended.

• At least 2 GB free disk space for project files and dependencies.

• Keyboard, mouse, and display for development and browser testing.

Software Requirements:

• Windows operating system.

• Java Development Kit (JDK) 17 or compatible Java environment.

• Apache Maven Wrapper (mvnw.cmd).

• Spring Boot 4.0.8.

• Apache Tika 3.2.3 dependencies.

• Web browser such as Google Chrome.

• Java-compatible IDE/editor (optional).

• Spring Boot web application with HTML/CSS/JavaScript frontend.

6. IMPLEMENTATION

Module- 1,2,3

Text analytics, Rabin–Karp, LCS, edit distance, Jaccard similarity, custom data structures, and heap-based ranking.

RabinKarp.java

src > main > java > plagiarism > algorithms > RabinKarp.java > { } plagiarism.algorithms

```java
package plagiarism.algorithms;

import plagiarism.datastructures.CustomLinkedList;

// Finds exact occurrences of a phrase (pattern) inside a larger text,
// using a rolling hash so each shift is checked in roughly O(1) time
// instead of re-comparing the whole pattern character by character.
public class RabinKarp {

    private static final int BASE = 256;             // number of possible characters
    private static final int MODULUS = 1000000007;    // large prime, reduces hash collisions

    public static CustomLinkedList<Integer> search(String text, String pattern) {
        CustomLinkedList<Integer> matches = new CustomLinkedList<>();

        int n = text.length();
        int m = pattern.length();
        if (m == 0 || n < m) {
            return matches;
        }

        long patternHash = 0;
        long textHash = 0;
        long highestPower = 1; // BASE^(m-1) % MODULUS, needed to remove the leading character later

        for (int i = 0; i < m - 1; i++) {
            highestPower = (highestPower * BASE) % MODULUS;
        }

        // Build the initial hash for the pattern and for the first window of text.
        for (int i = 0; i < m; i++) {
            patternHash = (BASE * patternHash + pattern.charAt(i)) % MODULUS;
            textHash = (BASE * textHash + text.charAt(i)) % MODULUS;
        }

```

EditDistance.java

src > main > java > plagiarism > algorithms > EditDistance.java > { } plagiarism.algorithms

```java
package plagiarism.algorithms;

// Levenshtein edit distance: the minimum number of single-character
// insertions, deletions, or substitutions needed to turn one word
// into another. We use this to catch near-spelling variants that
// a plain equality check would miss (e.g. "organize" vs "organized").
public class EditDistance {

    public static int calculate(String wordA, String wordB) {
        int n = wordA.length();
        int m = wordB.length();

        int[][] dp = new int[n + 1][m + 1];

        // Base cases: turning an empty string into wordB (or vice versa)
        // costs one insertion/deletion per character.
        for (int i = 0; i <= n; i++) dp[i][0] = i;
        for (int j = 0; j <= m; j++) dp[0][j] = j;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (wordA.charAt(i - 1) == wordB.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1]; // characters match, no extra cost
                } else {
                    int substitute = dp[i - 1][j - 1];
                    int insert = dp[i][j - 1];
                    int delete = dp[i - 1][j];
                    dp[i][j] = 1 + Math.min(substitute, Math.min(insert, delete));
                }
            }
        }

        return dp[n][m];
    }
```

LCS.java

src > main > java > plagiarism > algorithms > LCS.java > { } plagiarism.algorithms

```java
package plagiarism.algorithms;

import plagiarism.datastructures.CustomLinkedList;

// Longest Common Subsequence: the longest sequence of words that
// appears in both documents IN THE SAME RELATIVE ORDER, but not
// necessarily consecutively. This is how we catch reordered or
// partially rearranged copying - plain word overlap (Jaccard)
// can't tell order apart, but LCS can.
public class LCS {

    public static int lcsLength(CustomLinkedList<String> tokensA, CustomLinkedList<String> tokensB) {
        String[] a = toArray(tokensA);
        String[] b = toArray(tokensB);
        int n = a.length;
        int m = b.length;

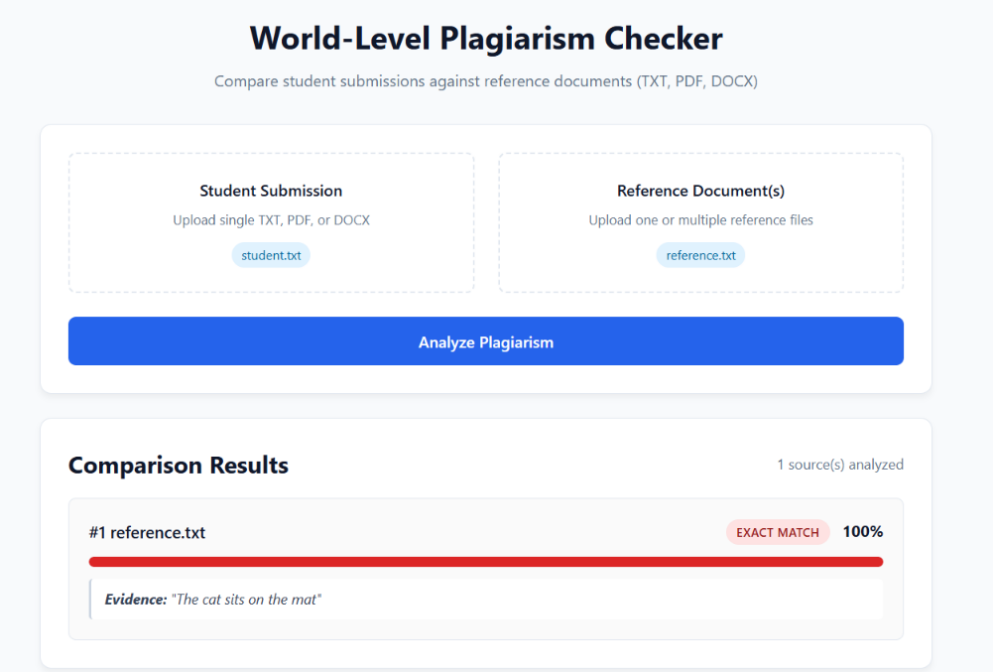
        // dp[i][j] = length of LCS between the first i words of a and first j words of b.
        int[][] dp = new int[n + 1][m + 1];

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (a[i - 1].equals(b[j - 1])) {
                    dp[i][j] = dp[i - 1][j - 1] + 1; // words match, extend the subsequence
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]); // take the better option
                }
            }
        }

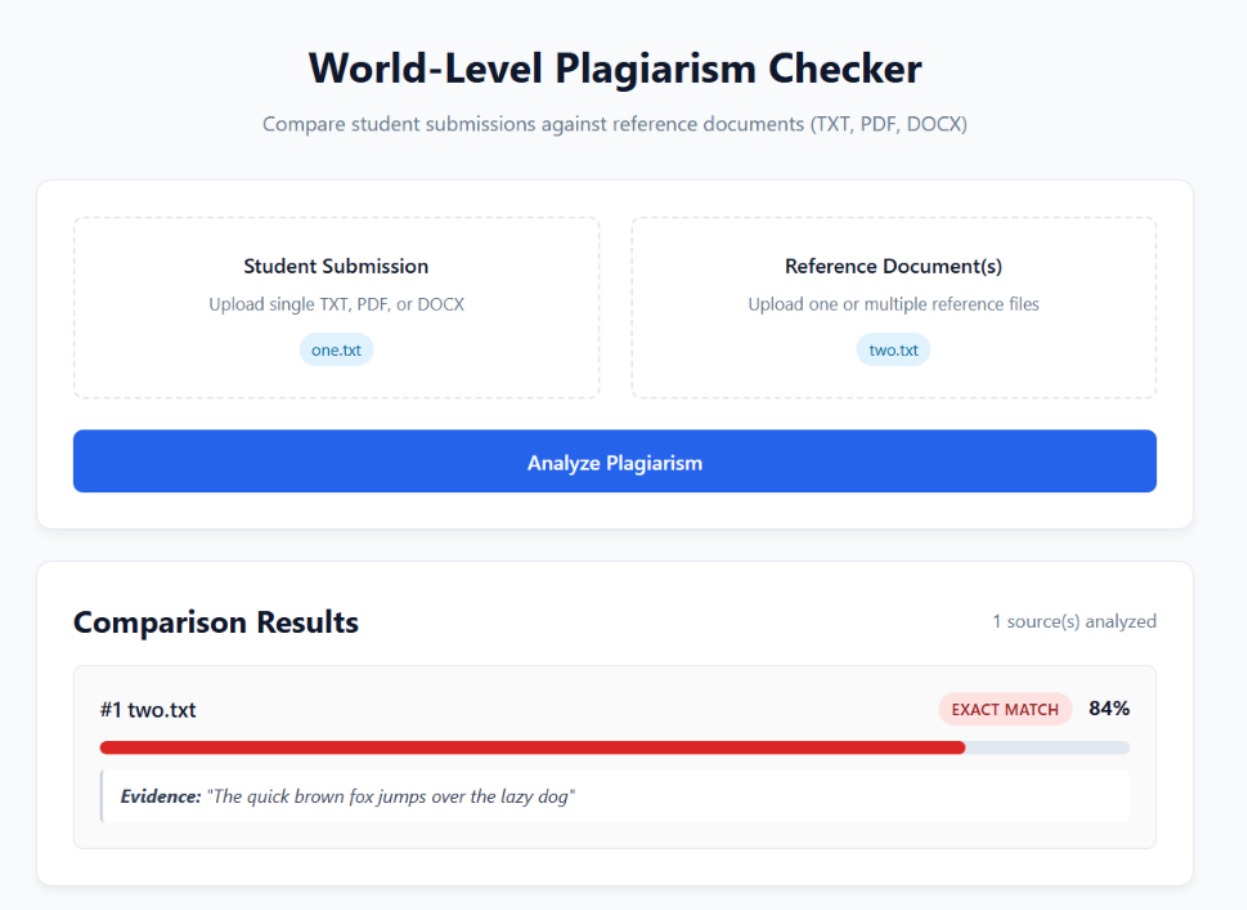
        return dp[n][m];
    }
```

7. RESULTS & DISCUSSION

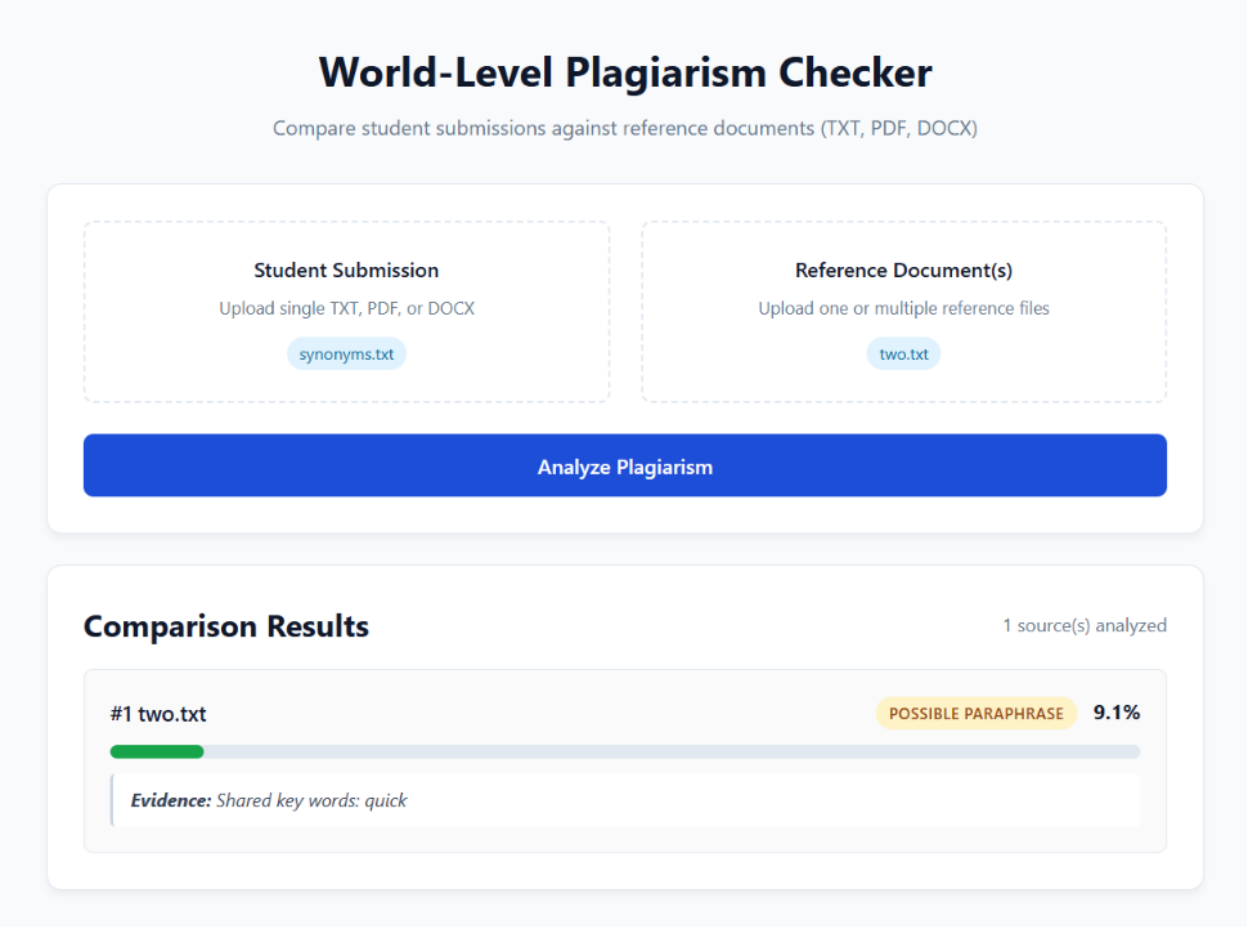
The implemented system was tested through automated tests and browser-based comparisons. The final automated test suite contained 10 tests and passed with zero failures, errors, or skipped tests. Browser testing confirmed the main classification behavior.



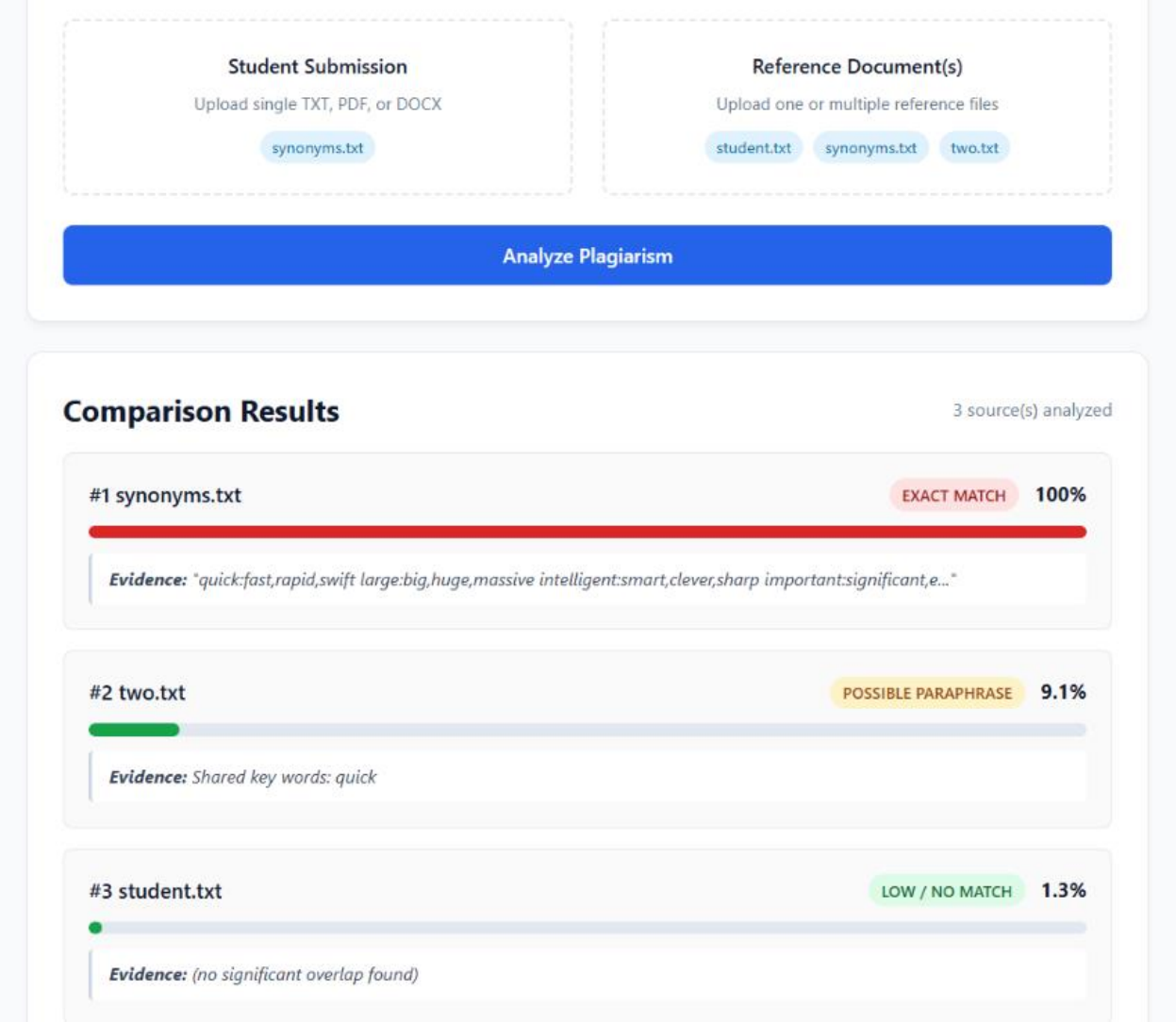
Exact Match: The same document text produced Exact Match with 100% similarity.



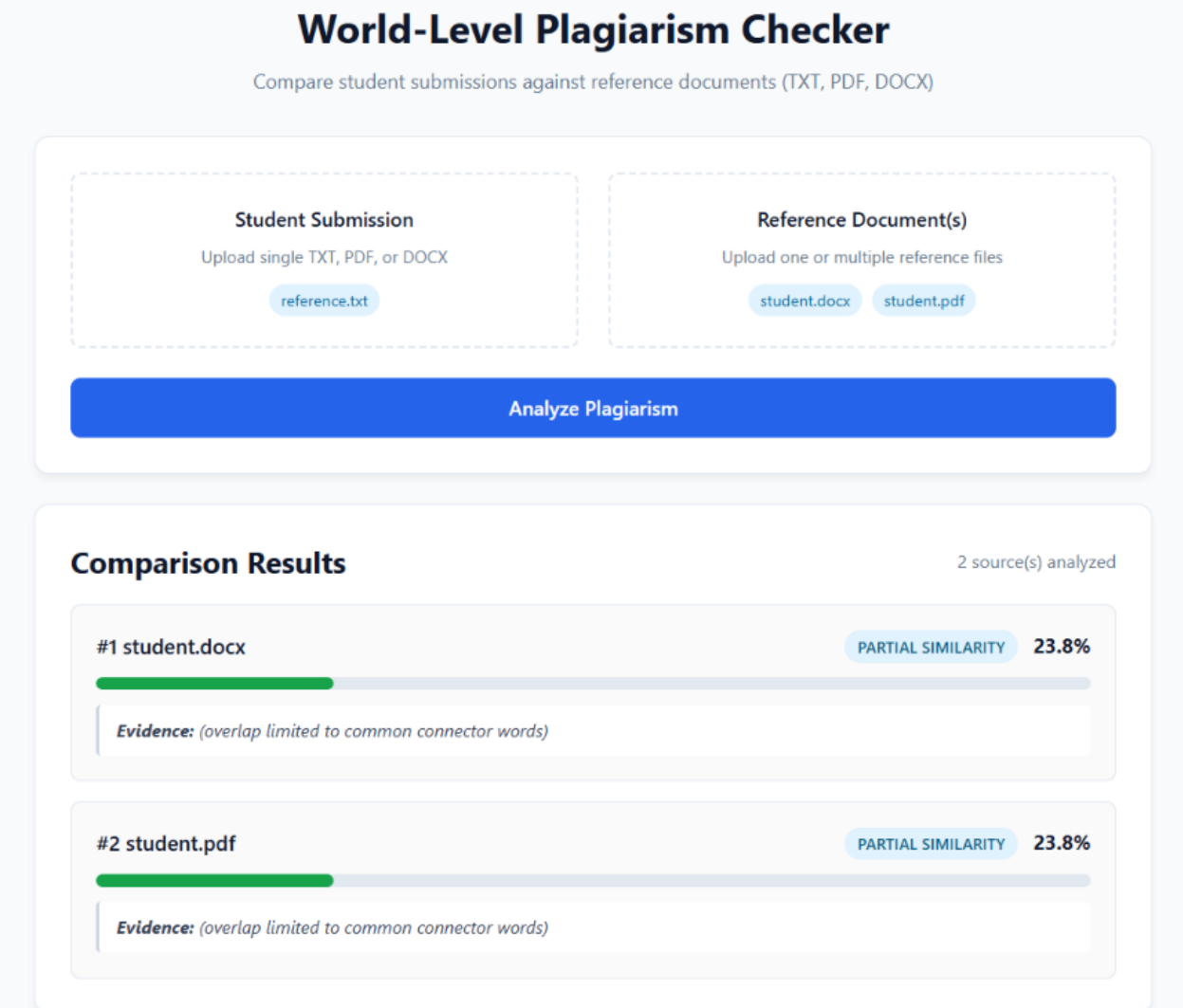
Reordered Words: The reordered-word example produced Reordered Words with approximately 76.2% similarity. Evidence displayed shared keywords such as through, ball, mouse, played, garden, and chased.



Possible Paraphrase: A synonym-based comparison produced Possible Paraphrase, confirming that the local synonym dictionary participates in classification.



Multiple References: The browser accepted multiple reference files and displayed ranked results for each source.



File Formats: TXT, PDF, and DOCX comparisons were successfully verified. Apache Tika extracted document text before it entered the DSA engine.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The World-Level Plagiarism Checker successfully demonstrates the use of Data Structures and Algorithms in a practical document comparison application. The project combines custom data structures and established string/similarity algorithms with a Spring Boot web interface. It supports multiple reference documents, common document formats, similarity percentages, match classifications, evidence snippets, and input validation. Testing confirmed that exact, reordered, synonym-based, and low-overlap comparison scenarios can be handled by the implemented engine.

Future Scope

- Add OCR support for scanned and image-only PDF documents.
- Expand the synonym corpus and improve contextual paraphrase analysis.
- Add sentence-level and passage-level highlighting.
- Support additional document formats and larger documents.
- Add persistent report storage and downloadable similarity reports.
- Improve the frontend with charts and visual comparison of matched passages.
- Introduce a larger local reference corpus for broader plagiarism detection.
- Improve performance through indexing, caching, and optimized similarity operations.

9. REFERENCES

1. Turnitin — https://www.turnitin.com/

2. Grammarly Plagiarism Checker — https://support.grammarly.com/hc/en-us/articles/115000091452-Plagiarism-Checker-user-guide

3. Quetext Plagiarism Checker — https://www.quetext.com/plagiarism-checker

4. Scribbr Plagiarism Checker — https://www.scribbr.com/plagiarism-checker/

5. Scribbr Plagiarism Check guide — https://help.scribbr.com/hc/en-us/articles/39232872423319-How-does-the-Scribbr-Plagiarism-Check-work

6. Copyleaks Plagiarism Checker API — https://docs.copyleaks.com/concepts/products/plagiarism-checker-api

7. Copyleaks Detect Plagiarism in Text — https://docs.copyleaks.com/guides/authenticity/detect-plagiarism-text/

8. Apache Tika Project Information — https://tika.apache.org/project-info.html

9. Apache Tika Download — https://tika.apache.org/download.html